UNIVERSITÀ DEGLI STUDI DI MILANO BICOCCA

Intro to Neural Networks

Lecture 12

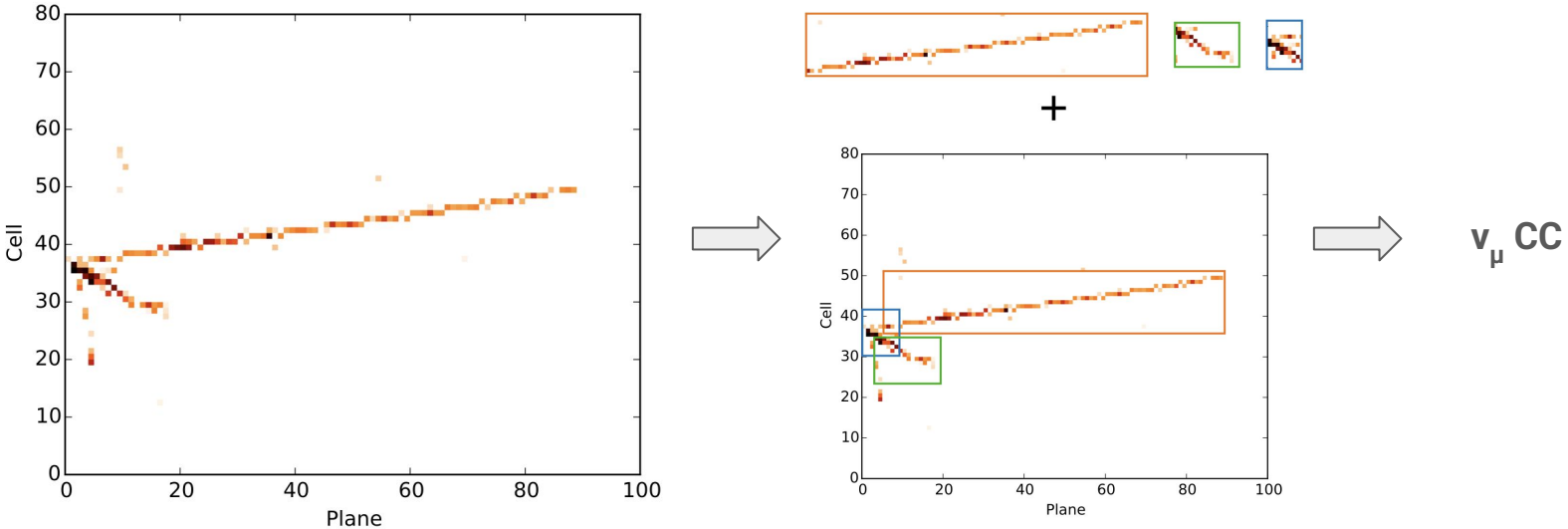
Course of:

Statistics and data analysis

Federico De Guio - Martino Borsato

Limitations of linear models

- **Problem**: a linear model considers **each feature independently** and regresses the weight w with which it contributes to the label
- But often individual low-level features (e.g. pixels in an image) are not meaningful. **What matters is the relationship between them**
- Example: To recognize a $\nu_\mu + n \rightarrow \mu + p$ interaction, we need to look at parts and the relationship between parts



Feed Forward Neural Networks

- James, cap.10
- Prince, cap.3, 4

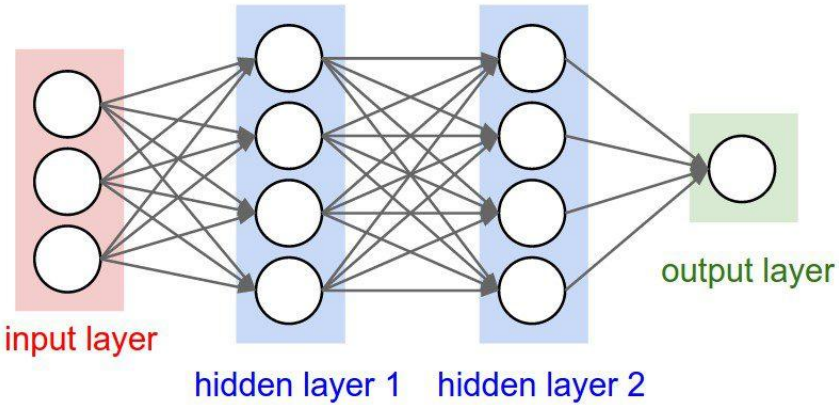
What is a ANN?

- A **mathematical model** able to approximate with high precision a **generic multi-dimensional function**:

$$f : R^n \rightarrow R^m : y = f(x) \longrightarrow \text{ANN}(x) = \hat{y}$$

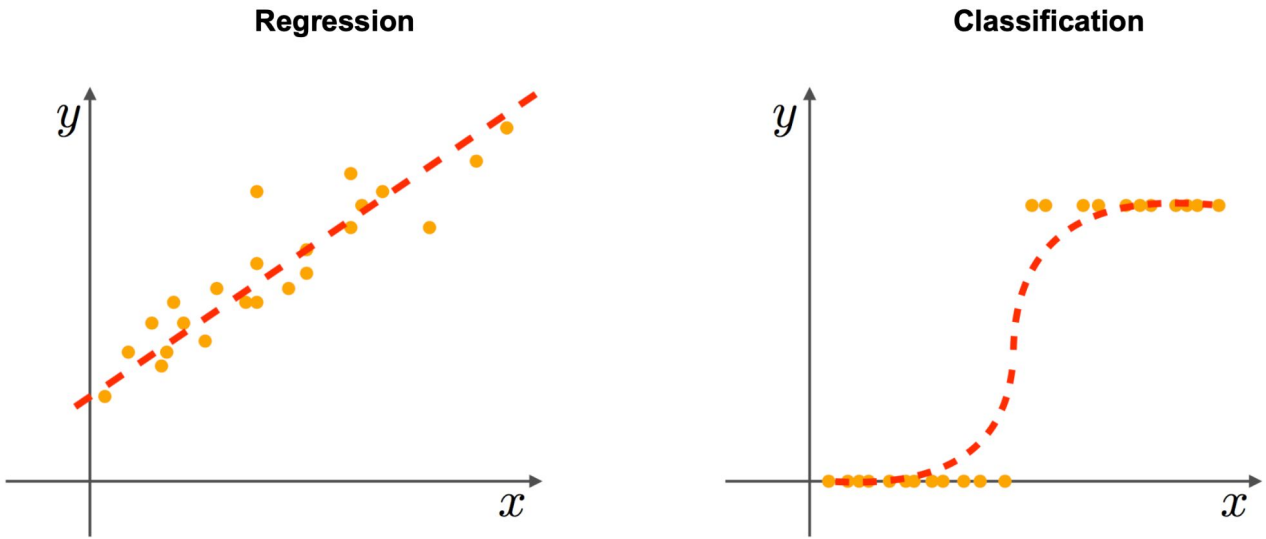
- Composition of functions (**layers**) connected in chains and described by **graphs**

- **Characteristics**:
  - Interconnected group of **identical computation units**
  - Collective **actions performed in parallel** by the neurons
  - The net structure dynamically evolves during the training
  - **Non-linear response**
  - Potentially complex multi-layer topologies



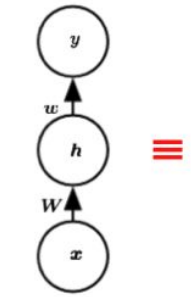
What problems can an ANN solve?

- **Regression:** model the relationship between variables
  - Linear vs non-linear regression, one vs multi-dimensional
- **Classification:** recognize and categorize the inputs
  - Two vs multi-category problems

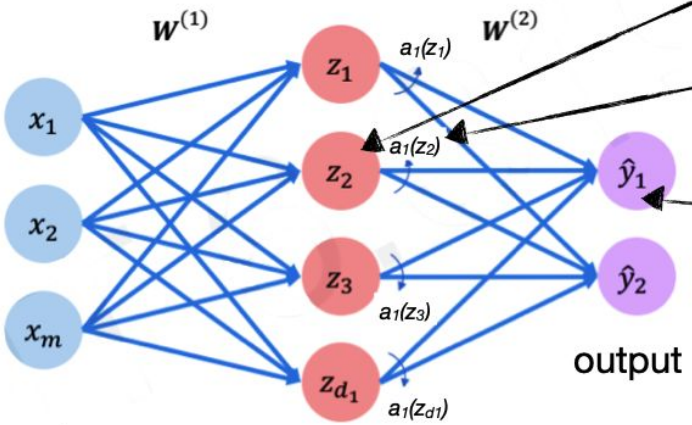


The feed forward NN

- Have **hierarchical structures**: inputs enter from the left and flow to the right + no closed loops
- How they work:
  - Each **input is multiplied by a weight**
  - The **weighted values are summed**
  - A **bias is added**
  - The result is **passed to an activation function**



FF-ANN
acyclic direct
graph



MLP with 1 hidden layer

$$z_i = w_{0i}^{(1)} + \sum_{j=1}^{m} x_j w_{ji}^{(1)}$$ synaptic sum

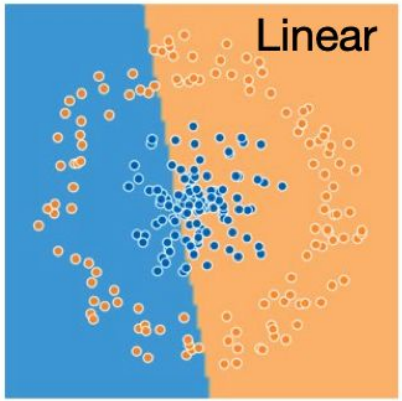
$$a_1(z_i)$$ activation

$$\hat{y}_i = a_2\left(w_{0i}^{(2)} + \sum_{j=1}^{d_1} a_1(z_j) w_{ji}^{(2)}\right)$$

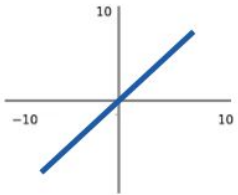
- NN behaviour determined by:
  - network topology (#layers, size of each layer, …)
  - weights w_{ij}
  - activation function of each layer

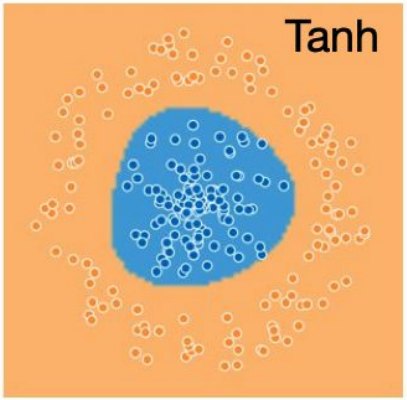
Linear vs non-linear activation functions

- **Non-linear activations** allow to learn complex and non-linear patterns

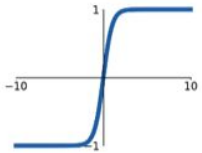


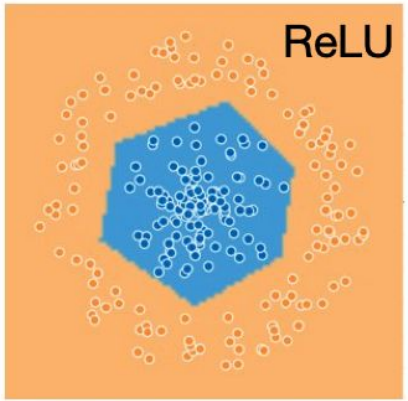
$$a(z) = z$$



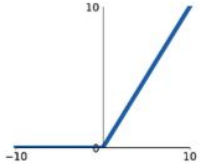


$$a(z) = \tanh[z]$$





$$a(z) = \max[0,z]$$

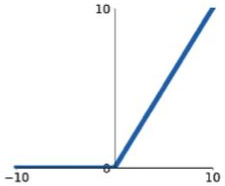


Choice of the activation function for the **hidden layers**

- In general, any **continuous and differentiable function** would be fine. In practice some functions work better than other for specific ANN architectures...

ReLU

$\max(0, x)$

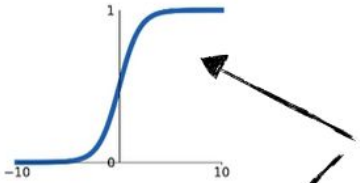


ReLu is the most popular:

- allows non linear dynamics
- faster convergence of the NN because doesn't saturate
- no vanishing gradient problem

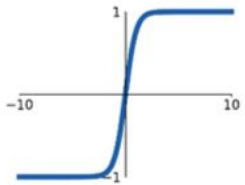
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



tanh

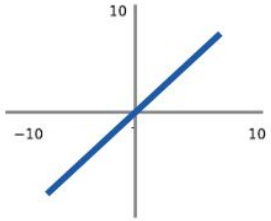
$\tanh(x)$



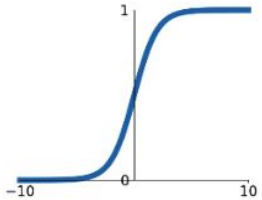
Should not be used in general for dense and convolutional layers:

- gradient vanishes away from x=0 → vanishing gradient problem
- sigmoid has output not centered in zero → affects SGD dynamic (zig-zag instabilities)

Choice of the activation function for the **output layer**



- **Identity (linear):** standard choice for regression tasks

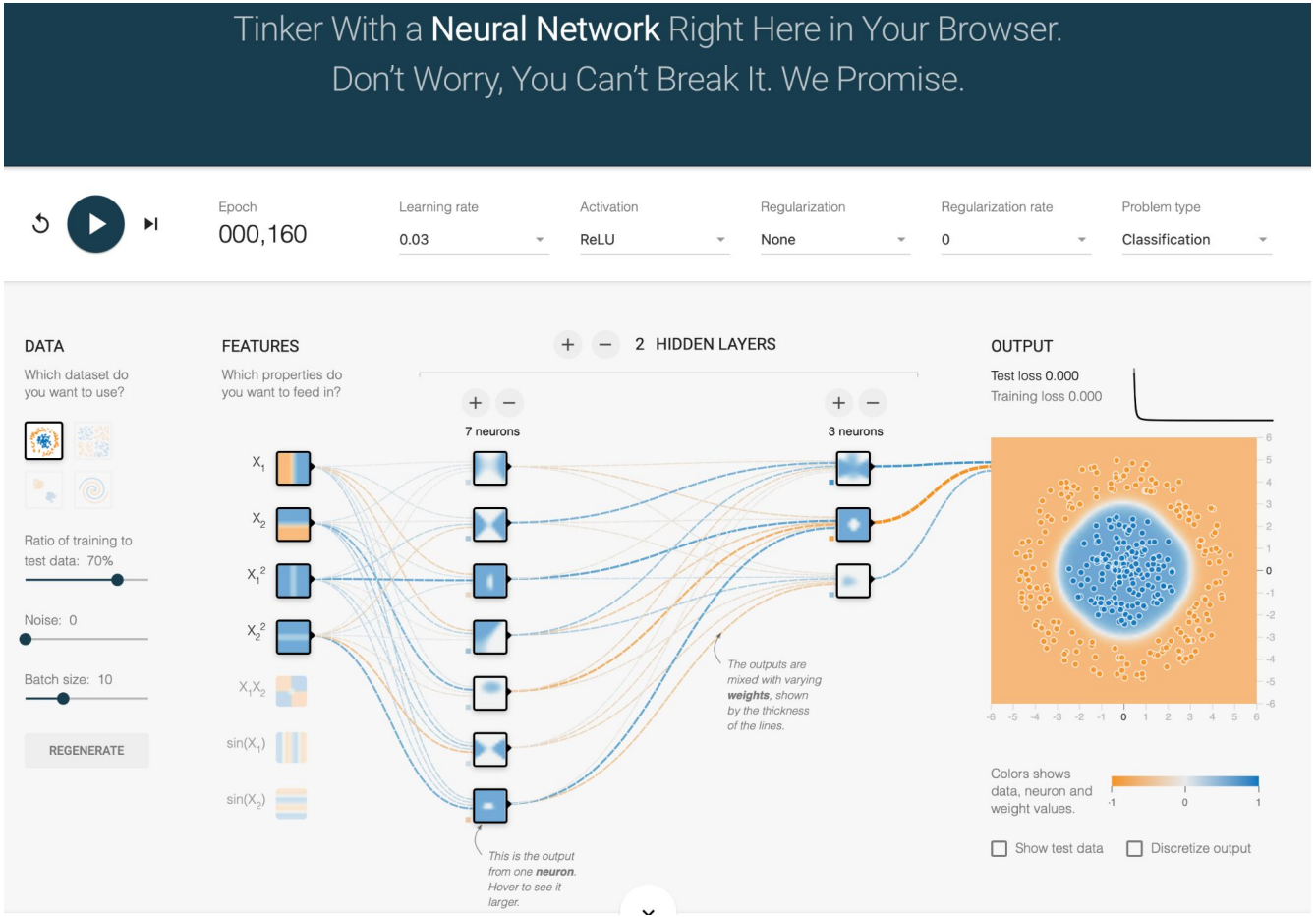


- **Sigmoid:** typically used in binary classification problems (2 classes) with a single output neuron or multilabel (multiple mutually inclusive classes) or sometime when the output features are numbers in (0,1)

$$y_i = \frac{e^{z_j}}{\sum_{j=1}^{n} e^{z_j}}$$

- **Softmax: $R^n \rightarrow [0,1]^n$**
  - soft version of the argmax output
  - often used in multi-class classification tasks (with mutually exclusive classes) - output of each neuron $\in$ (0,1) and interpretable as a probability ($\sum y^i = 1$)

Try yourself on https://playground.tensorflow.org



Tensorflow playground I

- **Problem A:**
  - Navigate your web browser to the TensorFlow Playground
  - Select the checkerboard pattern for the data, which is known as the **XOR** dataset
    - The data is sampled from a 2D probability density distribution represented by (x1, x2)
    - The regions x1, x2 >0 and x1, x2 <0 have a target value y = +1 and are shown as blue data points, while the regions x1 > 0, x2 < 0 and x1 < 0, x2 > 0 have a target value of y=−1 and are shown as orange data points.
  - First, select a linear model with no hidden layers
    - For the features, select the two independent variables x1 and x2
  - Can you fit the data with this linear model? Why or why not? What happens if you add the feature x1*x2?
  - Now, return the features to just (x1, x2) and start adding hidden layers
  - What's the smallest neural network (least number of layers and least number of neurons per layer) you can create that fits the training data "perfectly" (i.e. a training loss <0.001)?
  - What is the corresponding test loss?

Tensorflow playground II

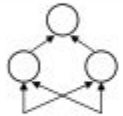
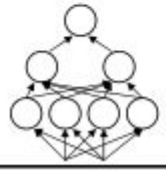
- **Problem B:**
  - Navigate your web browser to the TensorFlow Playground

  - Select the spiral pattern for the data. Using all the features available, find a solution that fits the training data.
  - What training and test error do you achieve?
  - Is this a low bias/high variance or high bias/low variance model? How do you know?

  - Using only (x1, x2), find a solution that fits the training data. What training and test error do you achieve?
  - Is this a low bias/high variance or high bias/low variance model? How do you know?

ANN as universal approximators

- It can be demonstrated that a feed-forward netw **number of neurons** with **non-linear activations** (subsets of R^n, under mild assumptions on the ac

Structur	Decision regions	Shapes
	sub-spaced delimited by hyperplanes	
	convex regions	
	arbitrary shaped regions	

$$F(x) = \sum c_i a(w_{0i} + \mathbf{w}^t \mathbf{x})$$

$$\int_{R^n} ||f(x) - F(x)||_p dx < \epsilon$$

IMPORTANT: the theorem doesn't say anything about the effective possibility to learn in a simple way the parameters of the model, all the DNN practice boils down to finding optimal and efficient techniques to solve this problem ...

The training process: learning the parameters

- Training consists in **adjusting the parameters according to a given cost function** (loss)
- The loss is a differentiable function that measures the **distance** wrt the prediction
  - **Weights and biases:** randomly initialized and "adjusted" using gradient descent with back-propagation
  - **Hyperparameters**: "adjusted" using heuristic approaches (manual trial&error, grid or random search, autoML, ...)
- For each event the output of the model $\hat{y}(x^{(i)})$ is calculated and compared with the expected target $y^{(i)}$ by means of an appropriate loss function:

$$L(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} L_i\left(y^{(i)}, \hat{y}^{(i)}(x^{(i)}\,|\,\mathbf{w})\right)$$

example: MSE

$$L_i\left(y^{(i)}, \hat{y}^{(i)}(x^{(i)}\,|\,\mathbf{w})\right) = \frac{1}{2}\left(y^{(i)} - \hat{y}^{(i)}(x^{(i)}\,|\,\mathbf{w})\right)^2$$

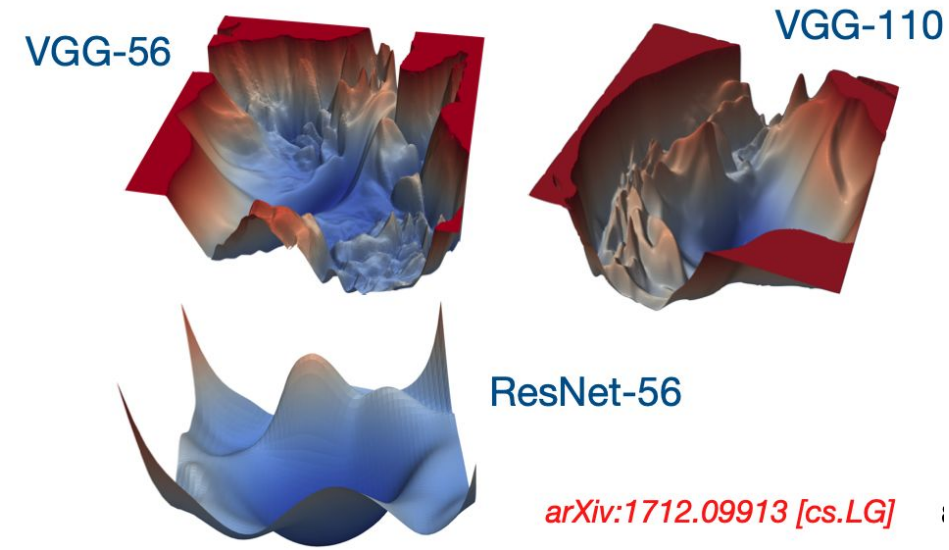
Target — Model output given ***w***

- Select the weights that minimize L: $\mathbf{w}^* = \underset{\mathbf{w}}{argmin}[L(\mathbf{w})]$
- → Find the minimum with GD techniques: $\mathbf{w}_{(t+1)} = \mathbf{w}_{(t)} - \eta \nabla_w L(T\,|\,\mathbf{w})$

Why going deep?

- One layer is in principle sufficient to approximate any function with arbitrary precision
- **Deep architectures are much more efficient** at representing a larger class of mapping functions

  - DNN require a **smaller number of parameters**
  - **Sub-features can be used in parallel** for multiple tasks within the same model
  - Overparameterization seems to have beneficial effects in smoothing the loss function → **easier convergence**



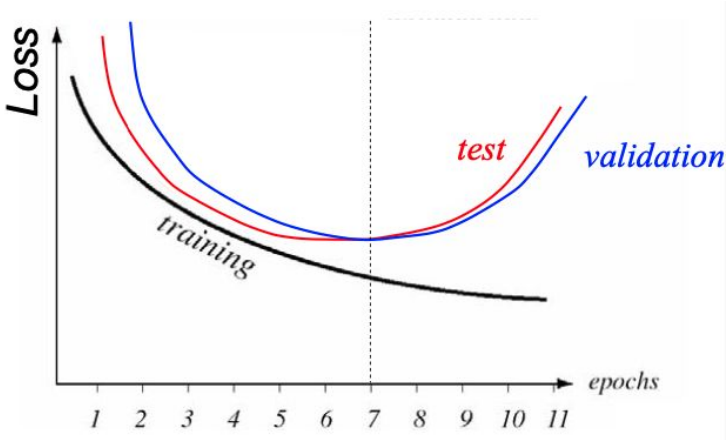
Learning curves

- At the **beginning** of the training phase the weights are randomly initialized → **the error on the training set is typically large**

- During the learning, the error decreases and **reaches a plateau** that depends on:
  - The size of the training set
  - The NN architecture
  - The initial value of the weights
  - …

- The effectiveness of the learning process is visualized with the **learning and accuracy curves**

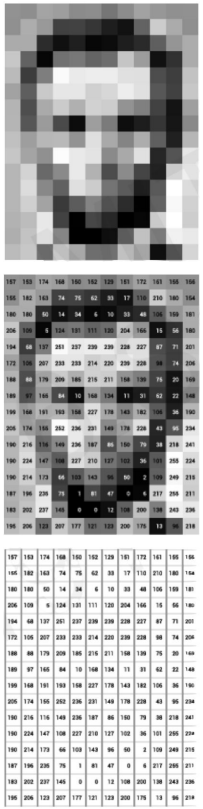
- **Overfitting** can be an issue: network too specialized on a given problem



Many different ANN architectures

- FFNN are **universal models**
  - However too much flexibility can results in arbitrarily complex models
  - Hard to optimize a huge number of parameters
  - hard to achieve a good level of generalisation

- → **Task independent priors** are introduced in modern DNN architectures
  - Priors are inferred from general structures observed in data
  - One example are **Convolutional Neural Networks which operate directly on images**
  - **More about CNNs next week!**



Convolutional NN is one of these specific DNN architecture designed to excel in image recognition tasks → will see details next week

Example…

The simplest architecture: sequential model (FFNN)

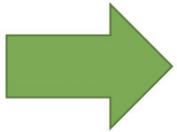
Keras

- Define a sequential model

```
model = Sequential()
mode.add(Dense(32, input_dim=784))
model.add(Activation('relu'))
model.add(Dense(10))
model.add(Activation('softmax'))
```

- Compilation

```
model.compile(optimizer='rmsprop',
              loss='binary_crossentropy',
              metrics=['accuray'])
```



- Training

```
model = model.fit(data, one_hot_labels,
epoch=10, batch_size=32)
```

- Predition

```
Y = model.predict(X)
```

Why Keras:
- User friendly with high-level APIs
- Quick to get started
- Coding less lines for machine learning model construction/training/testing

Case study

- Diagnose the diabetes disease based on generic information on the patient
- The available info is a table such as:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
545	8	186	90	35	225	34.5	0.423	37	1
471	0	137	70	38	0	33.2	0.170	22	0
21	8	99	84	0	0	35.4	0.388	50	0
46	1	146	56	0	0	29.7	0.564	29	0
755	1	128	88	39	110	36.5	1.057	37	1

- Here is the full dataset
- Let's give it a try!

Hands on…

Exercise 1: Build a **FFNN to classify sig vs bkg** events

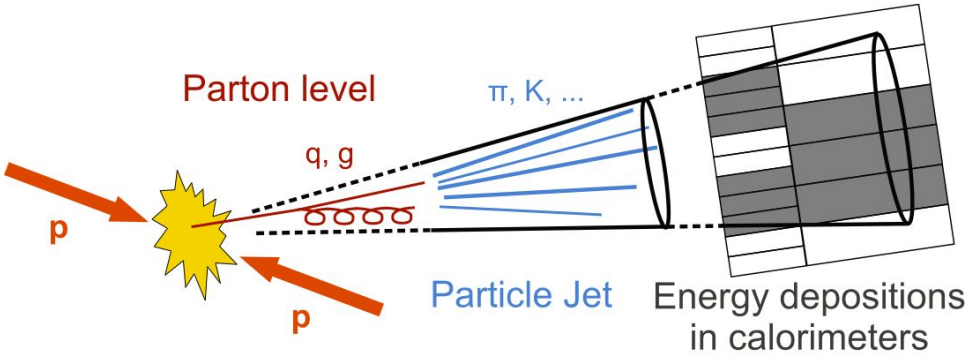
- This is the continuation of yesterday's exercise → use the same dataset as yesterday
- **Compare the results of the FFNN against the ones obtained with a tree based approach**
  - Which one does better and why?

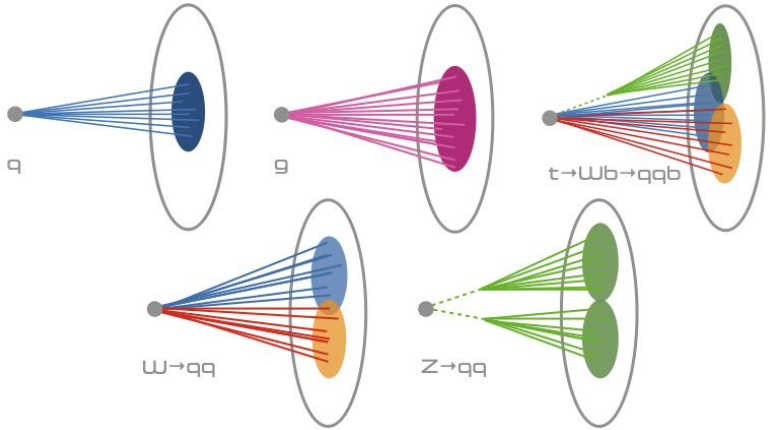
- **Goal:** distinguish exotic Higgs events from bkg involving top quarks
- The dataset can be accessed directly with the following syntax:

```
from sklearn.datasets import fetch_openml
# Load the Higgs dataset
higgs_data = fetch_openml(name='higgs', version=1, as_frame=True)
df = higgs_data.frame
```

Exercise 2: **Classification of particle jets** - multiclass classification

- **Jet tagging** at the Large Hadron Collider is a critical data analysis technique used to identify the originating particle of a jet
  - A jet is a collimated spray of particles produced in high-energy collisions
- Except for quark- (“q”) and gluon-initiated jets (“g”), all other jets in the dataset have at least one "prong"
  - The analysis of the structure of the jet provides info on the jet type
- **→ Implement a ML approach and perform multiclass classification**
  - The available features are energy correlation functions (1305.0007, 1609.07483).
  - The others are the jet mass, $\sum_z \log z$ where the sum is over the particles in the jet and z is the fraction of jet momentum carried by a given particle, and the overall multiplicity of particles in the jet.





Exercise 2: **Classification of particle jets** - multiclass classification

- How to access the dataset:

```
from sklearn.datasets import fetch_openml
# Load the jet dataset
data = fetch_openml("hls4ml_lhc_jets_hlf", parser="auto")
X, y = data["data"], data["target"]
```

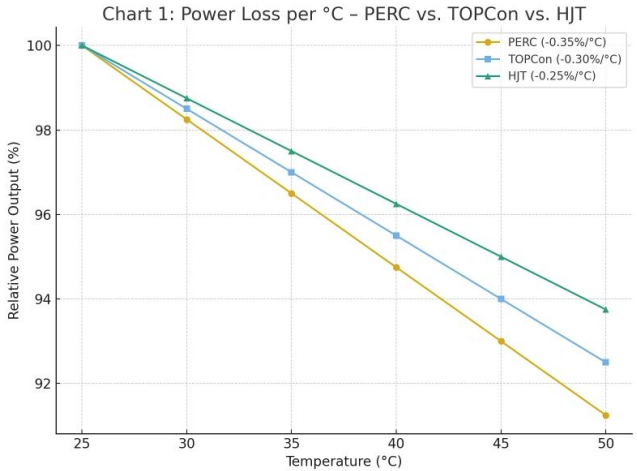
Exercise 3: Regression on **solar power data**

- The efficiency of solar panels **decreases as the operating temperature** of the panel **increases**
  - Order 5% every 10 degrees

$$P = \eta(T) \cdot G \cdot A$$

efficiency irradiance area

- Two datasets available showing data over 30 days from a real solar power plant:
  - Power generation data
  - Weather sensor data



- The idea: **predict the generated power**
  - Train a linear model and use it as a reference
  - Does a BDT improve the accuracy of the prediction?
  - Does a FFNN improve over a BDT?
  - What metric can be used to compare the various models?
  - What feature contributes?